

RTL Design Sherpa

RAPIDS beats on the Genesys 2 The FPGA System: Harness, Builds, Flow -- Version 1.1

September 2026

Table of Contents

1 Document Information....8
1.1 Revision History....8
1.2 How to read this book....9
2 The System....10
2.1 What it is....10
2.1.1 Figure 1.1: The bench....10
2.2 The board....11
2.3 The two USB chips....11
2.4 The stack, top to bottom....12
2.5 The design point....13
3 The Harness....14
3.0.1 Figure 2.1: The harness block map....14
3.1 The host front-end....14
3.2 The launch path: stage everything, then GO....15
3.3 Stimulus and checking....16
3.4 The run timer and the on-chip verdict....16
3.5 Memories the DUT owns....17
3.6 Measurement....18
3.7 The debug variant....18
4 One RTL, Several Bitstreams....20
4.1 The rule....20
4.1.1 Figure 3.1: One source tree, several bitstreams....20

4.2 The knobs..... 21
4.3 The variants and why each exists.....21
4.4 The design points..... 23
4.5 What a build costs and where it lives.....23
5 Host Tools and Flow.....25
5.1 Make targets.....25
5.2 The host tools..... 25
5.3 One measured cell, end to end.....27
5.3.1 Figure 4.1: From a host command to a report cell.....27
5.4 Simulation equals board.....28
5.5 Reprogram between campaigns.....28

List of Figures

Figure 1.1: The bench....10
Figure 2.1: The harness block map....14
Figure 3.1: One source tree, several bitstreams....20
Figure 4.1: From a host command to a report cell....27

List of Tables

Table 1: Table 0.1: Document information....8
Table 2: Table 0.2: Revision history....8
Table 3: Table 1.1: The part and what the harness uses of it....11
Table 4: Table 1.2: The stack....12
Table 5: Table 2.1: The three regions....14
Table 6: Table 2.2: Stimulus and checking....16
Table 7: Table 2.3: What the run timer compares, per half....17
Table 8: Table 2.4: Memories....17
Table 9: Table 3.1: The build knobs....21
Table 10: Table 3.2: The build variants....21
Table 11: Table 3.3: The design points....23
Table 12: Table 4.1: Make targets....25
Table 13: Table 4.2: Host tools....25

List of Waveforms

No waveforms in this document.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

1 Document Information

Table 0.1: Document information

Field	Value
Title	RAPIDS beats on the Genesys 2 – The FPGA System
Version	1.1
Date	2026-10-15
Scope	The `rapids_beats` characterization system: board, harness, build variants, host flow
Not in scope	The RAPIDS core itself (HAS/MAS under `projects/components/dma-ip/rapids/docs/`); the measured numbers (`reports/perf/README.md`)
Status	Current; written against the tree at the BUG-009 close (commit 37455c992), run timer added 2026-10-15 (commit 32d8a986b)

1.1 Revision History

Table 0.2: Revision history

Version	Date	Change
1.0	2026-09-29	First edition. Written after the 256-bit / 4 KB design point (report v2.0) and the BUG-009 close (report v2.2), so it describes the observers, monitors-in, 16 KB and ILA variants as actually built this month.
1.1	2026-10-15	The harness gains `char_run_timer`, the run timer and CRC verdict

Version	Date	Change
		shared with the STREAM harness: Figure 2.1 adds the block, a new section and Table 2.3 describe what it compares per half, Figure 4.1 shows it closing with the meters, and the host now requires its PASS beside the golden match.

1.2 How to read this book

Chapter 1 is the bench: what is plugged into what. Chapter 2 is the bitstream around the DUT, block by block, and the launch path a measurement takes. Chapter 3 is the reason there is more than one bitstream, and what each one buys. Chapter 4 is the flow: make targets, the host tools, and how a JSON row becomes a report cell. Every diagram is a Graphviz source under `assets/graphviz/`, rendered to PNG by `regenerate_all_graphviz.sh`.

RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

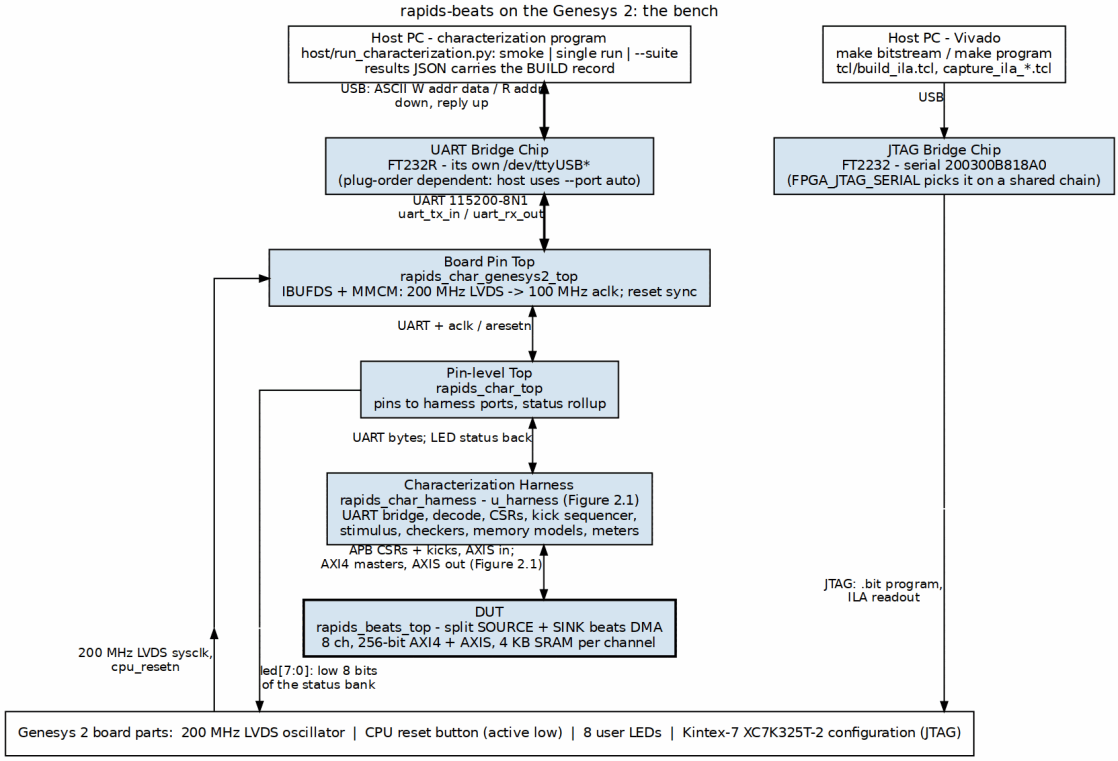
2 The System

2.1 What it is

`rapids_beats` is an on-chip characterization system. A synthesizable harness surrounds the RAPIDS beats DMA (split into a read-only SOURCE engine and a write-only SINK engine behind one APB) with seeded pattern generators, CRC checkers, descriptor and semaphore memories, a programmable memory-latency model and four per-cycle bus meters, and exposes the whole control surface to a host over one UART link. The host stages a configuration, fires one GO, and reads back a window of counts that bracket exactly the transfer that ran. Every configuration is golden-validated: the CRC the sink wrote to memory and the CRC the source produced on its stream are compared against a software model of the same LFSR pattern.

The point of the exercise is the number, not the demo: utilization per interface, per cycle, under conditions the host sets (channels, beats, burst length, memory latency, backpressure, channel schedule). Those numbers are the performance report; this book is the apparatus.

2.1.1 Figure 1.1: The bench



The bench: host, two USB chips, the FPGA

Source: 01_board_and_host.dot

2.2 The board

Digilent Genesys 2, Kintex-7 XC7K325T-2 (ffg900). The harness clock is 100 MHz: the board supplies a 200 MHz LVDS system clock, and `rapids_char_genesys2_top` turns it into the single-ended 100 MHz `aclk` through an IBUFDS and an MMCM. Everything downstream of that clock, the UART bridge, the decode, the harness and the DUT, is board-agnostic and was carried over unchanged from the Nexys A7 flow.

Why the Genesys 2 at all: the Artix-7 100T on the Nexys A7 is at its timing limit for RAPIDS beats above four channels at a 512-bit datapath. The Kintex-7 325T has roughly three times the LUTs and a faster speed grade, so the full eight-channel geometry closes 100 MHz with margin, and the instrumented variants of Chapter 3 fit at all. The `BOARD` knob still selects either target; the Genesys 2 defaults to eight channels, the Nexys to four.

Table 1.1: The part and what the harness uses of it

Resource	XC7K325T-2	Note
Slice LUTs	203,800	the observers build of the 256-bit design point uses about 76,000
Block RAM tiles	445	the same build uses 44
User LEDs	8	the low 8 bits of the harness status bank
Reset	red CPU reset button, active low	synchronized inside the pin top

2.3 The two USB chips

The Genesys 2 has two separate USB bridges, and the distinction matters every time a board is set up:

- The **FT2232** carries JTAG. Vivado programs the bitstream and talks to the ILA through it. Its serial, `200300B818A0`, is what `FPGA_JTAG_SERIAL` names so a two-board chain (a Nexys A7 sits on the same chain in this lab) programs the right part.

- The **FT232R** carries the UART. It enumerates as its own `/dev/ttyUSB*`, and which number it gets depends on plug order. The host tools therefore default to `--port auto` and probe every `ttyUSB` for the harness ID register rather than pinning a node.

The link is 115200-8N1 with an ASCII protocol: `W addr data` and `R addr`, one 32-bit AXI4-Lite access per line. That is slow, and the harness is built so that slowness never lands inside a measurement (Chapter 2, the launch path).

2.4 The stack, top to bottom

Table 1.2: The stack

Layer	Module	Role
Pin top	`rapids_char_genesys2_top`	LVDS clock in, MMCM, reset sync, LEDs; instantiates the pin-level top
Pin-level top	`rapids_char_top`	pins to harness ports; historically the host front-end, now pins plus status only
Harness	`rapids_char_harness`	UART to AXI4-Lite bridge, region decode, harness CSRs, APB master, kick sequencer, generators, checkers, memories, latency model, meters, optional observers
DUT	`rapids_beats_top`	SOURCE (memory to AXIS) and SINK (AXIS to memory), 8 channels, one APB, merged MonBus egress

The host front-end used to live in the pin-level top. It was moved into the harness on 2026-09-23 so that simulating the harness exercises the board's own launch path: the cocotb testbench drives the same UART bytestream the host does, and the same decode, CSRs and kick sequencer run. That is what makes "sim equals board" a checkable claim rather than a hope (Chapter 4).

2.5 The design point

Since 2026-09-29 the DUT is built at 256-bit AXI4 and AXIS (RAPIDS has one `DATA_WIDTH` for both) with 128 beats of SRAM per channel, 4 KB. The earlier 512-bit, 256-deep (16 KB) point is what report versions 1.0 to 1.5 measured. The line rate is therefore 3.20 GB/s per direction: 32 bytes per beat at 100 MHz.

The bitstream reports its own geometry. Harness CSR 0x004, `BUILD`, carries bytes per beat, channel count, log2 of the SRAM depth and the three instrument flags, and the host reads it on first contact and takes its byte and bandwidth arithmetic from there. A host started with a channel count the bitstream does not have refuses to run. Every results file carries that `design` record, so a number can never be attributed to the wrong build.

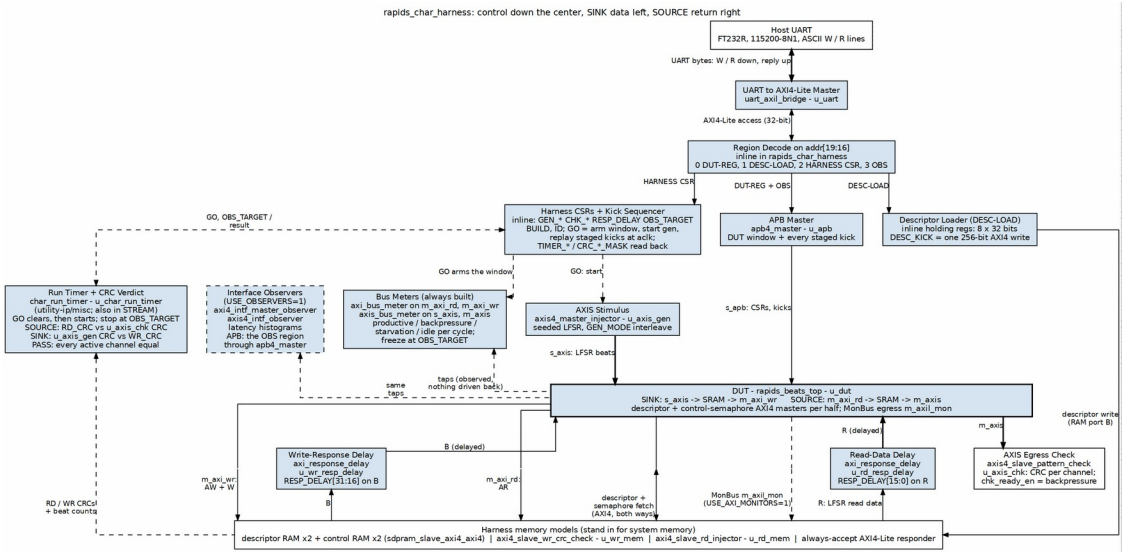
RTL Design Sherpa · Learning Hardware Design Through Practice GitHub · Documentation Index · MIT License

---

3 The Harness

`rapids_char_harness` is the bitstream minus the pins. It has three jobs: give the host a control surface, feed and check the DUT with synthesizable stimulus, and measure without perturbing. Figure 2.1 is the block map; the sections below take the blocks in the order a transfer meets them.

3.0.1 Figure 2.1: The harness block map



The harness: front-end, stimulus, memories, DUT, measurement

Source: 02_harness_blocks.dot

3.1 The host front-end

`uart_axil_bridge` turns the UART bytestream into a single 32-bit AXI4-Lite master. A decode on address bits 19:16 splits that master into three regions:

Table 2.1: The three regions

Region	Word address	What is behind it
DUT-REG	`0x0_0000`	an APB window into the DUT through `apb4_master`: address bits 12:0 are the APB byte address, so the SOURCE register space at 0x0000, the SINK space at 0x1000 and the per-channel kick windows are all reachable

Region	Word address	What is behind it
DESC-LOAD	0x1_0000	eight 32-bit holding words assemble one 256-bit descriptor; DESC_KICK issues a single-beat AXI4 write of it into the SOURCE or SINK descriptor RAM; DESC_STATUS reports the write landed
HARNESS CSR	0x2_0000	generator, checker, memory and monitor control, the status and CRC readback, the meter counts, the run timer and verdict (0x160-0x180), BUILD, ID

The observers build adds a fourth window for the observers' own APB configuration and readback; the host tools address it by name through the generated register map, never by offset.

Registers are reached by name everywhere. The harness CSR map and the DUT's map come out of PeakRDL as Python register maps (`rtl/rapids_harness_csr_regmap.py`, `rtl/rapids_harness_desc_regmap.py`, and the DUT's `rapids_regmap.py`), and `rapids_char_io.py` writes fields by name. Offsets churn; names do not.

3.2 The launch path: stage everything, then GO

UART is slow, a few hundred microseconds per access. If the measurement window opened at the first CSR write and closed at the last, the numbers would measure the link, not the DMA. Two mechanisms make the numbers trustworthy:

1. **Atomic launch.** Every CSR, every descriptor and every per-channel kick is staged first. A single write to GO then arms the meter window, starts the AXIS generator and fires all the staged kicks on-chip within a few aclk cycles. The kick sequencer holds the kicks; the host never times anything.
2. **Deterministic close.** The host stages OBS_TARGET, the number of productive beats the transfer will produce on the completion interface. The meters freeze the cycle after that count is reached, so the window brackets exactly the transfer, independent of the DUT's internal idle signalling. Earlier builds keyed the close on system_idle and left the window open until the host read it, which diluted utilization toward zero.

Between runs the host pulses CHANNEL_RESET on both halves. That clears the schedulers and descriptor engines; it does not reach the SRAM controller, which is why BUG-009's poisoned states survived it and why a campaign that must not inherit history starts from a reprogram.

3.3 Stimulus and checking

Everything that produces or consumes data is on-chip and seeded, so the board and the simulator see the same bytes.

Table 2.2: Stimulus and checking

Block	Side	What it does
`axis4_master_injector`	SINK ingress (`s_axis`)	LFSR data from `GEN_SEED`, `GEN_NBEATS` per channel, `GEN_CHMASK`, `GEN_BPP` beats per packet, `GEN_TDEST`; `GEN_MODE.INTERLEAVE` round-robins the active channels one beat at a time instead of finishing one channel before starting the next
`axis4_slave_pattern_check`	SOURCE egress (`m_axis`)	per-channel CRC of what arrives; `CHK_CTRL.chk_ready_en` is the egress backpressure knob
`axi4_slave_rd_injector`	backs `m_axi_rd`	LFSR read data for the SOURCE, a CRC of what it served
`axi4_slave_wr_crc_check`	backs `m_axi_wr`	per-channel CRC of what the SINK wrote
`axi_response_delay` x2	in front of both slaves	`RESP_DELAY`: hold R and B for a programmed number of cycles, the memory-latency knob of the latency sweeps

The golden model on the host, `rapids_char_golden.py`, runs the same LFSR and computes the CRCs a correct DMA must produce. PASS is anchored on the data path: the sink's write CRC and the source's egress CRC must equal the model.

3.4 The run timer and the on-chip verdict

`char_run_timer` (`u_char_run_timer`, from `projects/components/utility-ip/misc/rtl/`) is the same module the STREAM harness uses, so both DMAs are timed and judged the same way. A RAPIDS run tests one half, and the timer is fed that half's input and output side, selected by `KICK_CFG.HALF`:

Table 2.3: What the run timer compares, per half

Half	Input side	Output side	Stop count
SOURCE	`RD_CRC`, the LFSR source behind `m_axi_rd`	`CHK_ACT_CRC`, the egress pattern check on `m_axis`	egress beats
SINK	`GEN_EXP_CRC`, the AXIS generator	`WR_CRC`, the write CRC check behind `m_axi_wr`	write beats

- **Clear and start:** GO clears the timer and starts it one cycle later.
- **Stop:** the output-side beat count reaching `OBS_TARGET`, the same target that closes the meter window. `TIMER_CYC_LO/HI` freeze there.
- **Pass:** latched 32 cycles after the stop. Every channel that ran must have both CRCs valid and equal; a channel that never ran neither passes nor fails.
- **Stamps:** `TIMER_R_*` / `TIMER_W_*` hold the cycle of the first and last input-side and output-side beat.

The host requires `TIMER_STATUS.pass` and every active channel in `CRC_MATCH_MASK`, as well as the golden match. A board run that fails only the on-chip check is a real mismatch between what entered a half and what left it. `BUILD.RUN_TIMER` says whether a bitstream carries the timer; the host skips the on-chip check on one that does not.

3.5 Memories the DUT owns

Table 2.4: Memories

Block	Count	Role
descriptor RAM (`sdpram_slave_axi4_axi4`)	2	one per half; the DUT reads descriptors over `m_axi_desc`, the host writes them through DESC-LOAD
control semaphore RAM (`sdpram_slave_axi4_axi4`)	2	one per half; the control-write master writes and the control-read master reads the same store, so a doorbell write is

Block	Count	Role
		observable by a gate read
always-accept AXI4-Lite responder	1	terminates the DUT's `m_axil_mon` MonBus egress so it never stalls

3.6 Measurement

Four bus meters are always built, one per data interface: `axi_bus_meter` on the read and write AXI4 masters, `axis_bus_meter` on the AXIS in and out. Each classifies every cycle of the window into productive, backpressure, starvation or idle, and the host turns those into utilization and effective bandwidth. They are cheap, and they are the source of truth for report sections 1 to 6.

Two optional instruments, each a build variant of Chapter 3:

- **Observers** (`USE_OBSERVERS=1`): the shared `axi4_intf_master_observer` on the AXI masters and `axis4_intf_observer` on the AXIS links. Passive taps on the same wires the meters watch, with their own APB window, per channel utilization and read/write latency histograms (`HIST_MAX_OUTSTANDING=32` so the latency sweeps do not lose samples). They are what measured STREAM's five knobs on this DUT (report section 7). `OBS_ENABLE_MON_TAPS` additionally arms their MonBus event taps.
- **Monitors-in** (`USE_AXI_MONITORS=1` with `GEN_MON=1`): the in-core AXI and descriptor monitors inside the DUT and the per-channel completion and error emitters that give their packets an egress. On by default in a product; off here by default because the bare build is tuned to close eight channels, and on when the question is what the monitors cost.

3.7 The debug variant

`tcl/build_ila.tcl` defines `RAPIDS_CHAR_ILA`, under which the harness and the DUT's engines mark their observation-window nets `mark_debug`, and the script inserts an ILA on every marked net. The marks expand to nothing in every other build, so they cost the measured bitstreams no logic and no timing. `tcl/capture_ila_bug009.tcl` is the capture driver: program, arm on `r_go` or on a late AXI4-write beat count, drive one host run, write the trace as CSV. Its optional poison pre-run and no-reprogram modes exist because BUG-009 was history-dependent.

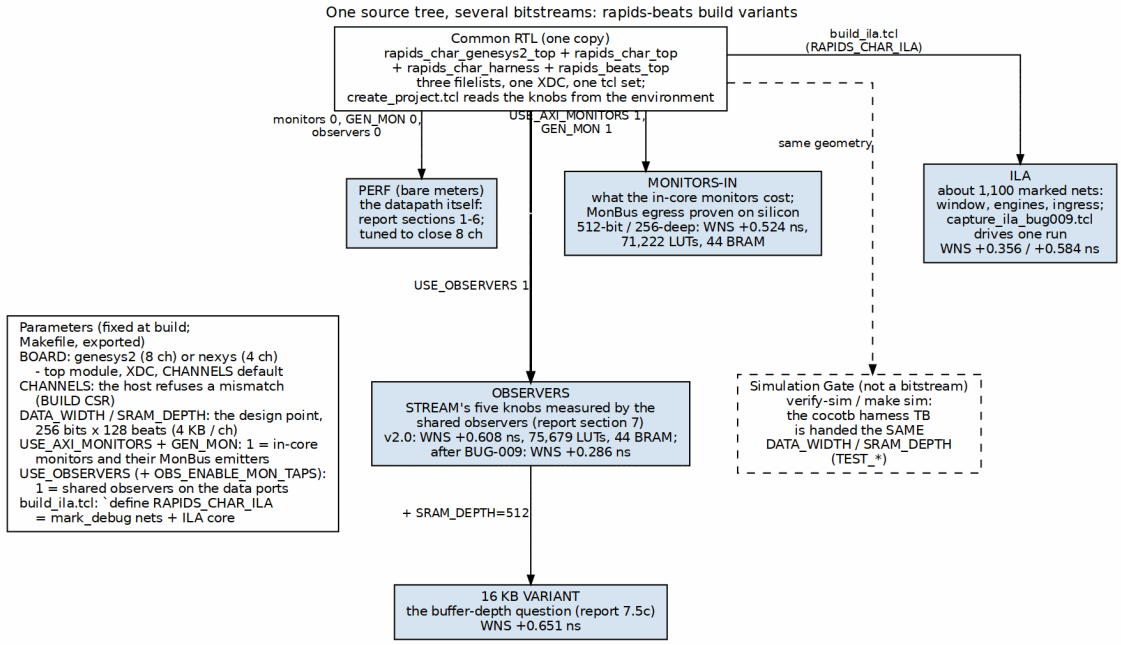
4 One RTL, Several Bitstreams

4.1 The rule

There is one harness, one pin top, one board top, one set of filelists and one XDC. Every bitstream this area has ever measured came from that source at a different set of parameters. Nothing is copied to make a variant: a variant is a line in the Makefile, exported into the environment, and `tcl/create_project.tcl` reads it there and hands it to Vivado as a generic or a define. The alternative, a near-copy of the harness per flavour, is how two designs drift apart while everyone believes they are measuring the same thing; STREAM retired exactly such a pair, and this area never had one.

The same knobs go to the simulator. `make verify-sim` and `make sim` pass `DATA_WIDTH` and `SRAM_DEPTH` to the cocotb harness testbench as `TEST_DATA_WIDTH` and `TEST_SRAM_DEPTH`, so the pre-bitstream gate simulates the geometry the bitstream will have, and a bug that only exists at 128 beats is not hidden by a 512-beat simulation.

4.1.1 Figure 3.1: One source tree, several bitstreams



One RTL, the knobs, the builds

Source: 03_build_variants.dot

4.2 The knobs

Table 3.1: The build knobs

Knob	Reaches the design as	Selects
`BOARD`	top module, XDC, default `CHANNELS`	`genesys2` (eight channels) or `nexys` (four)
`CHANNELS`	`RAPIDS_NUM_CHANNELS`	channel count, also handed to the host so the campaign and the bitstream cannot disagree
`DATA_WIDTH`, `SRAM_DEPTH`	`RAPIDS_DATA_WIDTH`, `RAPIDS_SRAM_DEPTH`	the design point; both readable back through `BUILD`
`USE_AXI_MONITORS`, `GEN_MON`	DUT generics	the in-core monitors and their MonBus emitters
`USE_OBSERVERS`, `OBS_ENABLE_MON_TAPS`	harness generics	the shared interface observers and their event taps
`tcl/build_ila.tcl`	`` `define RAPIDS_CHAR_ILA ``	the marked debug nets and an ILA core

The monitor knobs deliberately carry the same names STREAM's `build-*` Makefiles export, so a reader who knows one area's builds knows the other's.

4.3 The variants and why each exists

Table 3.2: The build variants

Variant	Knobs	The question it answers	Measured
Performanc	monitors	how fast the datapath is, measured by	report

Variant	Knobs	The question it answers	Measured
e (bare meters)	0, `GEN_MON` 0, observers 0	the four cheap meters; the bitstream is tuned to close eight channels at 100 MHz, so this is the reference point every other variant is compared against	sections 1 to 6, every design point
Monitors-in	`USE_AXI_MONITORS=1`, `GEN_MON=1`	what the in-core monitors cost in area and timing, and whether the DUT's MonBus egress works on silicon at all; also the first build after a monitor-lite update to check timing had not moved	512-bit, 256-deep, 8 ch (2026-09-28): WNS +0.524 ns, 71,222 LUTs, 44 BRAM tiles; all 172 campaign configurations identical to the bare build
Observers	`USE_OBSERVERS=1` (taps optional)	STREAM's five characterization knobs measured by the shared interface observers instead of the bare meters, with latency histograms; the passive taps must not change a single utilization cell, and they did not	256-bit, 4 KB, 8 ch: WNS +0.608 ns, 75,679 LUTs, 44 BRAM tiles (report v2.0); after the BUG-009 fixes WNS +0.286 ns
16 KB variant	observers build with `RAPIDS_SRAM_DEPTH=512`	whether the per-channel buffer depth is in the eight-channel latency knee (it is, report 7.5c) and in the single-channel window (it is not)	WNS +0.651 ns
ILA	`build_ila.tcl` on the bare	see the engine gates, the ingress allocation and the SRAM controller's counts during one run; the BUG-009	about 1,100 marked nets; WNS +0.356

Variant	Knobs	The question it answers	Measured
	build	traces	ns and +0.584 ns on the two builds this month

Two things follow from the table. First, the instrumented variants are not redundant with each other: observers and monitors watch from opposite sides of the DUT boundary, and each has a zero the other cannot see. Second, the bare build is where timing is tuned, so any variant that closes with less margin is reporting the cost of its instrument, not a regression of the datapath.

4.4 The design points

Table 3.3: The design points

Point	`DATA_WIDTH` / `SRAM_DEPTH`	Line rate	Reports
Original	512 / 256 (16 KB per channel)	6.40 GB/s	v1.0 to v1.5
Current	256 / 128 (4 KB per channel)	3.20 GB/s	v2.0 onward

Every utilization cell of the 256-bit build matched the 512-bit build at the same beat count, so the engines' per-beat behaviour is width-independent and every gigabyte figure simply halves. Two things the smaller buffer did change: a fixed 82-cycle ingress fill stall per run, and a single-channel write window of about 80 beats instead of about 93. Those are the report's findings; the point here is that the comparison was possible only because both points came from the same source at different generics.

4.5 What a build costs and where it lives

A full flow is synthesis, implementation and bitstream generation, ten to forty minutes on the lab host. `make bitstream` runs `verify-sim` first unless `BITSTREAM_SKIP_VERIFY=1`, refuses to start Vivado if the sink self-check fails, and writes `bitstream/rapids_char.bit` plus the timing, utilization and DRC reports under `reports/`. Bitstreams are never committed. The one or two worth keeping are copied out of the tree by `make keep` into the hold directory (`RDS_HOLD_DIR`), and `make program` falls back to that copy when the build directory is clean, saying which file it is programming every time.

5 Host Tools and Flow

5.1 Make targets

The flow Makefile in `flows-rapids-beats/` is the one entry point. It exports the knobs of Chapter 3 and drives Vivado, the simulator and the host tools with them.

Table 4.1: Make targets

Target	What it does
`make sim`	the cocotb harness self-check, sink and source, at the Makefile's `DATA_WIDTH` and `SRAM_DEPTH`
`make verify-sim`	the sink self-check only; the gate `make bitstream` runs first
`make bitstream`	synthesis, implementation, bitstream and reports (`BOARD=genesys2 USE_OBSERVERS=1` for the observers build)
`make program`	flash the board over JTAG, naming the file it programs
`make smoke`	a fast golden-validated check over UART, two channels, four beats, both halves
`make suite`	the full sweep, results to a JSON file
`make flow`	sim, bitstream, program, characterize

5.2 The host tools

Table 4.2: Host tools

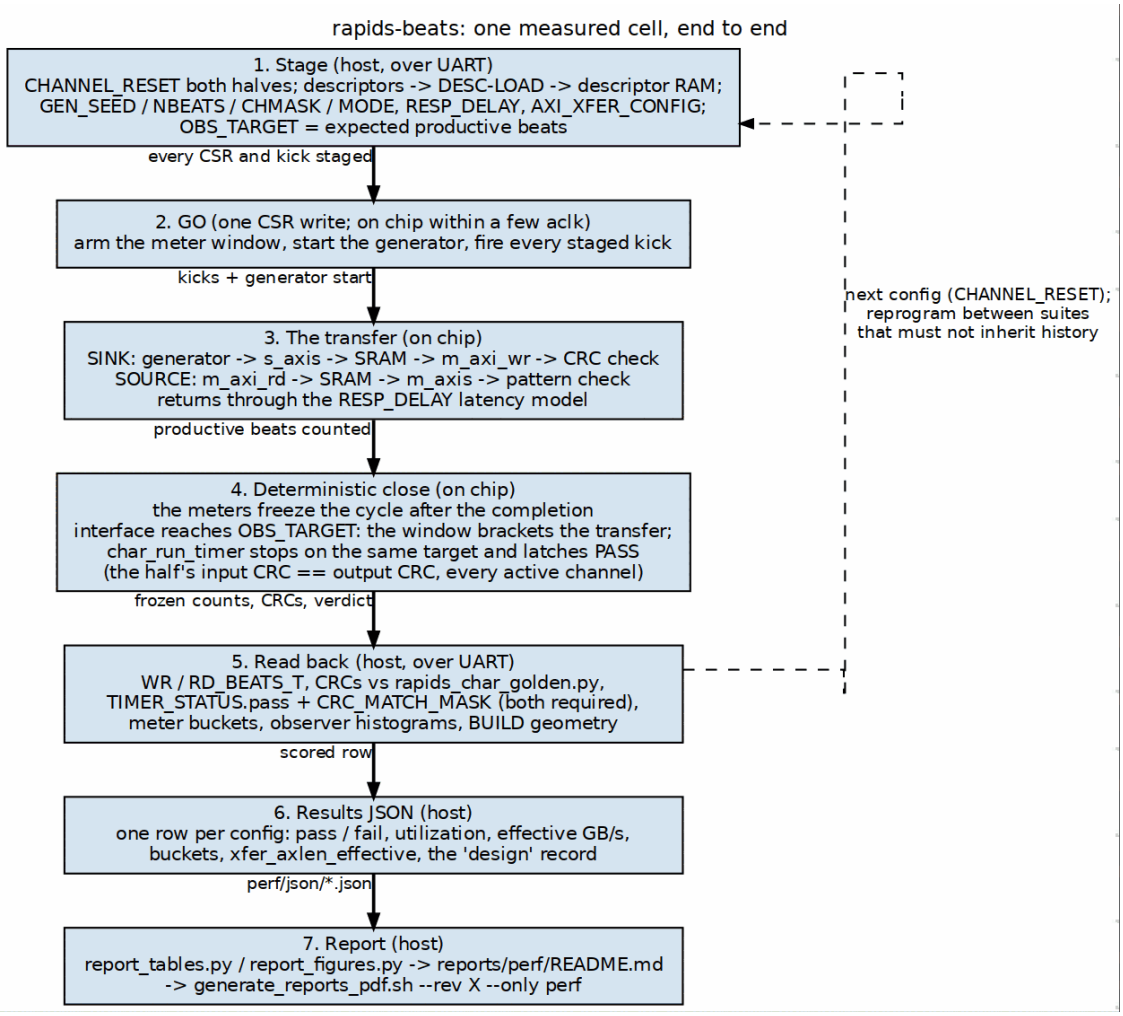
Tool	Role
`rapids_char_io.py`	the UART link and register access by name through the generated register maps
`descriptor_builder.py`	assembles the 256-bit descriptors the DESC-LOAD region writes
`rapids_char_golden.py`	the software LFSR and CRC model every run is validated against
`run_characterization.py`	the campaign driver: `--smoke`, a single

Tool	Role
	run (`--active`, `--beats`, `--xfer-axlen`, `--interleave`, `--backpressure`, `--sink-only` / `--source-only`), or `--suite` with `--suite-channels`, `--suite-beats`, `--suite-descs`, `--suite-xfer`, `--suite-delay`, `--suite-bp`, `--suite-seeds`; every row carries the `BUILD` record and the AxLEN the engines actually ran
`dump_status.py`	pretty-prints the status CSRs of a running board
`run_sink_once.py`	one sink run, for debugging by hand
`report_tables.py`, `report_figures.py`	rebuild the report's tables and PNG figures from any campaign's JSON files
`plot_char_reports.py`, `gather_size_sweep.sh`	the older plotting and gathering helpers

`run_characterization.py` reads `BUILD` on first hardware contact and refuses a `--channels` that does not match the bitstream. It warns when a requested AxLEN exceeds the build's cap (the SRAM depth minus one, since BUG-009) and records the effective value in every row, so a table never labels a 128-beat burst as 256.

5.3 One measured cell, end to end

5.3.1 Figure 4.1: From a host command to a report cell



The measurement path

Source: 04_measurement_path.dot

For each configuration the driver resets both halves, loads the descriptors, programs the generator and the knobs, stages the kicks and the productive beat target, writes GO, polls the completion counter, and scores: beat totals, CRCs against the golden model, the on-chip verdict from the run timer (Section 2), meter buckets, and on the observers build the histograms. The row goes into the results JSON. The report scripts turn the JSON files under `reports/perf/json/` into the tables and figures of `reports/perf/README.md`, and `reports/generate_reports_pdf.sh --rev X --only perf` turns that into the versioned DOCX and PDF.

5.4 Simulation equals board

The harness simulation drives the same UART bytestream through the same bridge, decode, CSRs and kick sequencer that the board runs, against the same generics. `dv/test_rapids_char_harness.py` carries the sink and source self-checks, and the environment knobs `TEST_NUM_ACTIVE`, `TEST_NUM_BEATS`, `TEST_XFER_AXLEN`, `TEST_RESP_DELAY_RD` and `TEST_RESP_DELAY_WR`, `TEST_GEN_INTERLEAVE` and `TEST_USE_OBSERVERS` reach the same registers the host's options do. A board finding is therefore reproduced by setting those knobs, not by writing a new test.

The limits are worth knowing. The self-checks run one transfer each, so a defect that needs history (a run that ends mid-segment, then a run that needs the whole buffer) is a board reproduction first and a macro-level test in the component area second. And the simulator's slave models drain at their own rhythm, so a race that depends on a partial allocation landing on a stale view was seen only on silicon, through the ILA variant.

5.5 Reprogram between campaigns

A wedged channel is cleared by a reprogram, not by `CHANNEL_RESET`. The campaign scripts that produce report data therefore program the board before each suite, and a smoke run is treated as history: it ends mid-segment, and the BUG-009 investigation showed what a following full-depth burst did with that.